

Getting Started with Python Testing Using Pytest

Pytest로 시작하는 파이썬 테스트

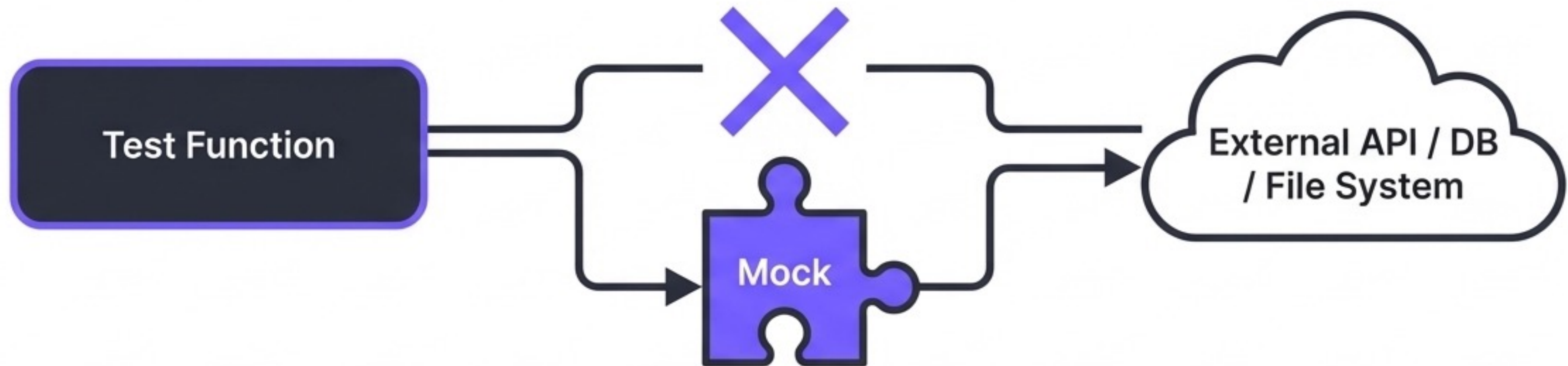
실전 테스트

테스트 심화 – Mock



4단계 : 외부 요인 차단, 순수 로직만 검증 (Mock)

실제 환경에 영향을 주는 위험한 로직은
가짜 객체(Mock)로 대체하여 완벽하게 고립시켜야 함

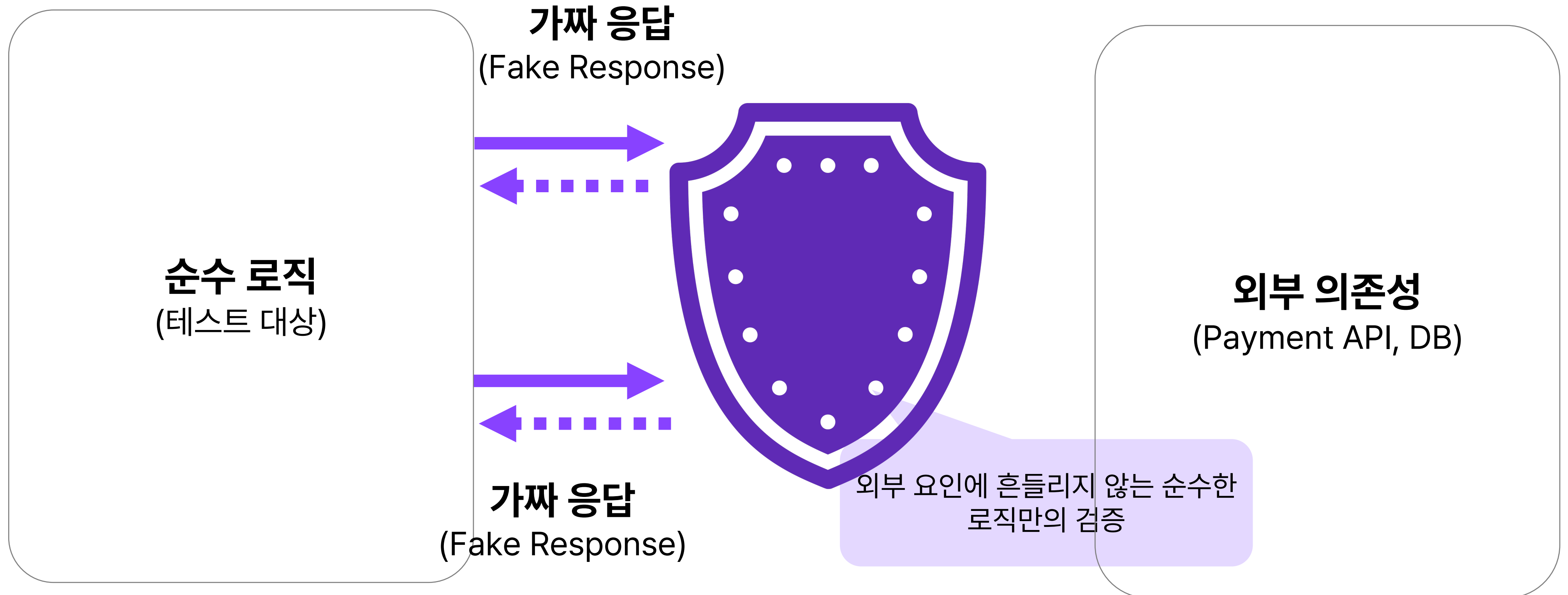


1. 대체: `unittest.mock.patch`를 사용해 특정 모듈이나 함수를 Mock 객체로 전환

2. 검증: `assert_called_once_with()`를 사용하여 해당 함수가 기대한 인자와 함께 정확히 호출되었는지 확인



의존성 격리 - Mocking의 원리





Mock 패밀리 : Mock · MagicMock · patch

Mock

```
from unittest.mock import Mock

# 호출 · 반환만 흉내
m = Mock(return_value=42)
m() # 42

len(m) # TypeError
```

최소형 가짜

호출·반환만 흉내내는 **기본 가짜**
매직 메서드(`__len__`, `__iter__`)는 **막힘**

MagicMock

```
from unittest.mock import MagicMock

# 매직 메서드까지 자동
mm = MagicMock()
len(mm) # 0
iter(mm) # OK

with mm: ... # OK
```

Mock + 매직 메서드

`len/iter/with` 등 **매직 메서드 자동 처리**
실무 디폴트 — `patch`가 자동 생성

patch

```
from unittest.mock import patch

# 실제 위치에 가짜를 꽂음
@patch("app.send_email")
def test_x(mock_send):
    ...

# mock_send → MagicMock
```

가짜를 꽂는 도구

실제 코드 위치에 **MagicMock 주입**
끝나면 **자동 원복** — 안전

핵심 · 직접 임포트하는 건 보통 **patch**, 그 결과 객체는 자동으로 **MagicMock**. **Mock**은 매직 메서드를 막고 싶을 때만



가짜로 바꾸기 : @unittest.mock.patch

```
from unittest.mock import patch

# app.send_email 을 가짜로 교체
@patch("app.send_email")
def test_signup(mock_send):
    app.signup("Elice@a.com")
    assert mock_send.called

# 가짜 응답값 지정
mock_send.return_value = True
```

```
@patch("가짜로 바꿀 위치")
def 테스트함수(mock):
```

인자: "사용되는 곳"

주입: MagicMock

효과: 실제 호출 차단

핵심 포인트

- 경로는 "쓰는 모듈" 기준으로 적기
- `return_value.side_effect` 로 가짜 응답 제어



가짜 응답 제어 : return_value vs side_effect

```
# 1) return_value — 항상 같은 값 반환
mock_api.return_value = {"ok": True}
mock_api("x") # {'ok': True}
mock_api("y") # {'ok': True} (동일)

# 2) side_effect — 호출마다 다른 응답
mock_api.side_effect = [
    "first", "second", "third"
]
mock_api() # 'first'
mock_api() # 'second'

# 3) side_effect — 예외 발생 시키기
mock_api.side_effect = TimeoutError
mock_api() # raise TimeoutError
```

return_value — 고정값. 정상 응답을 흉내낼 때
side_effect = [...] — 호출마다 순차 응답
side_effect = Exception — 예외 시뮬레이션
(네트워크 에러 등)

핵심 포인트

- 둘 다 있으면 **side_effect** 우선 (단, 함수가 None을 반환하면 return_value 사용)
- **side_effect**에 함수를 넣으면 인자에 따라 동적 응답도 가능



두 가지 사용법 : @patch vs with patch()

```
# 1) @patch — 함수 전체에 적용
@patch("app.send_email")
def test_signup(mock_send):
    app.signup("a@b.com")
    assert mock_send.called

# 2) with patch() — 블록 안에서만
def test_signup():
    real_setup() # 진짜 함수
    with patch("app.send_email") as m:
        app.signup("a@b.com")
        assert m.called
    real_check() # 다시 진짜
```

@patch — 함수 전체 패치, mock 자동 주입

with patch() — 블록 안만 패치, **as**로 mock 받음

공통: 범위 종료 시 자동 복구

핵심 포인트

- 함수 전체를 가짜로 → **@patch**
- 일부 구간만 가짜로 → **with patch()**
- 여러 곳을 패치할 땐 **@patch** 중첩 (인자 순서 = 아래에서 위)



호출 검증 메서드 비교

```
# 불리언 / 카운트
mock.called           # bool
mock.call_count       # int

# 호출 여부
mock.assert_called()
mock.assert_not_called()

# 인자까지 검증
mock.assert_called_with("x")
# → 마지막 호출만

mock.assert_called_once_with("x")
# → 1회 + 인자 일치
```

"한 번 이상 호출됐나?" → `assert_called()`

"이 인자로 호출됐나?" → `assert_called_with(...)`

"딱 1번, 이 인자로?" → `assert_called_once_with(...)`

"호출 안 됐나?" → `assert_not_called()`

핵심 포인트

- `_with` 붙으면 **인자 검증**, `once` 들어가면 **횟수까지 검증**
- 여러 번 호출 인자를 모두 보려면 `call_args_list`

테스트 심화 – 테스트 커버리지, 비동기 테스트

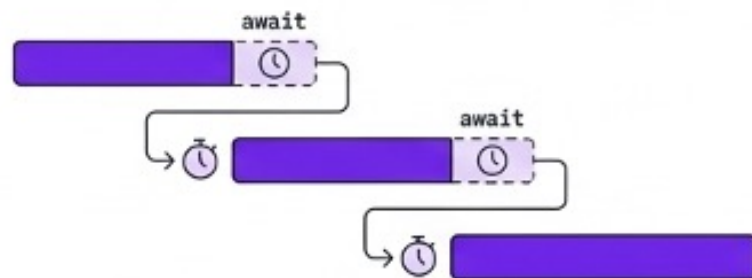
코드 신뢰성의 시각화 및 비동기 검증



`pytest-cov`

테스트 커버리지 측정

내 전체 코드 중 테스트가 수행된 비율을 수치와 시각적 리포트로 확인. 맹점을 파악하여 신뢰성 향상



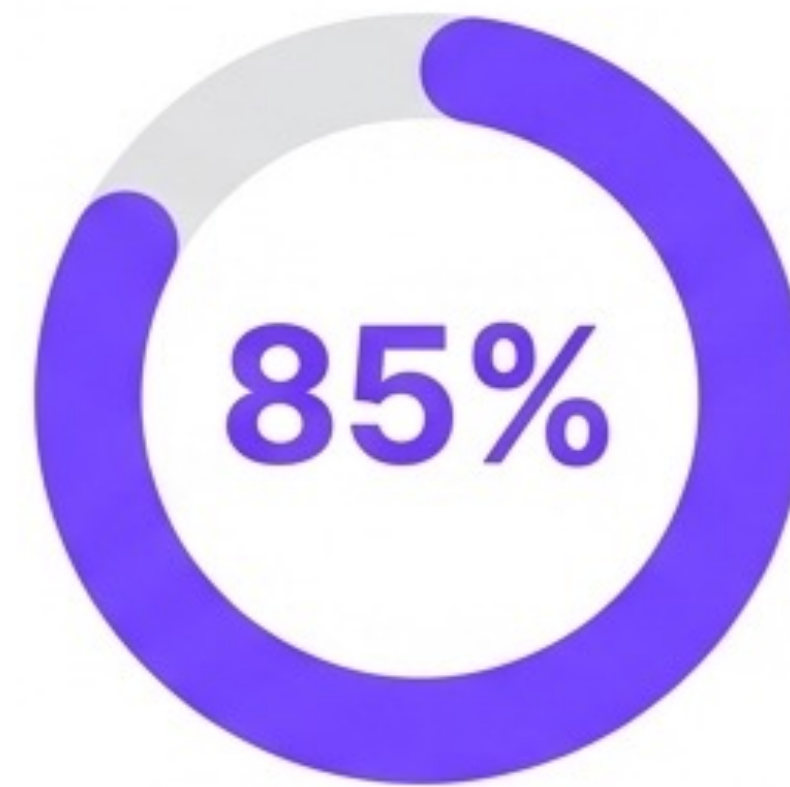
`pytest-asyncio`

비동기 로직 검증

async/await가 포함된 현대적인 비동기 함수의 동작을 정확하게 대기하고 검증

5단계 : 내 코드의 사각지대 찾기 (테스트 커버리지)

```
> pytest --cov=my_project
```



Stmts : 전체 실행 가능한
코드 문장 수

Miss 테스트가 한 번도
실행하지 않은 문장 수

Cover : 테스트 커버리지
비율

pytest-cov 옵션 치트시트

로컬 개발: 빠진 줄 즉시 확인

```
pytest --cov=myapp --cov-report=term-missing
```

HTML로 라인별 추적

```
pytest --cov=myapp --cov-report=term-missing --cov-report=html
```

CI 게이트 + Codecov 업로드

```
pytest --cov=myapp --cov-report=xml --cov-fail-under=80 --cov-branch
```

옵션	효과
term	터미널 요약만
term-missing	+ 빠진 줄 번호
term-missing:skip-covered	100% 파일은 표에서 숨김
html	htmlcov/ 폴더 — 라인별 클릭 추적
xml	coverage.xml — CI/Codecov 업로드용
json	JSON — 스크립트 처리용

옵션	효과
--cov=pkg	측정 대상 패키지 (반복 가능)
--cov-fail-under=N	N% 미만이면 실패 ★ CI 게이트
--cov-branch	라인 + 분기(if/else 양쪽) 측정
--cov-append	기존 .coverage에 누적 (병렬 합산)
--cov-context=test	어떤 테스트가 어떤 라인 탔는지
--cov-config=.coveragerc	설정 파일 지정

테스트 커버리지 결과 예시

```
pytest -v --cov=order_service --cov-report=term-missing
===== test session starts =====
platform linux -- Python 3.10.13, pytest-9.0.3, pluggy-1.6.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /home/elicer/project
plugins: asyncio-1.3.0, cov-7.1.0
asyncio: mode=strict, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 14 items

test_order_service.py::test_calculate_total PASSED [ 7%]
test_order_service.py::test_validate_order_success PASSED [ 14%]
test_order_service.py::test_apply_discount[10000-VIP-8000] PASSED [ 21%]
test_order_service.py::test_apply_discount[10000-GOLD-9000] PASSED [ 28%]
test_order_service.py::test_apply_discount[10000-NORMAL-10000] PASSED [ 35%]
test_order_service.py::test_calculate_total_invalid_item PASSED [ 42%]
test_order_service.py::test_validate_order_empty_items PASSED [ 50%]
test_order_service.py::test_validate_order_invalid_quantity PASSED [ 57%]
test_order_service.py::test_apply_discount_invalid_grade PASSED [ 64%]
test_order_service.py::test_checkout_success PASSED [ 71%]
test_order_service.py::test_checkout_calls_payment_api_with_discounted_amount PASSED [ 78%]
test_order_service.py::test_send_order_message PASSED [ 85%]
test_order_service.py::test_send_order_message_with_validation PASSED [ 92%]
test_order_service.py::test_send_order_message_with_validation_error PASSED [100%]

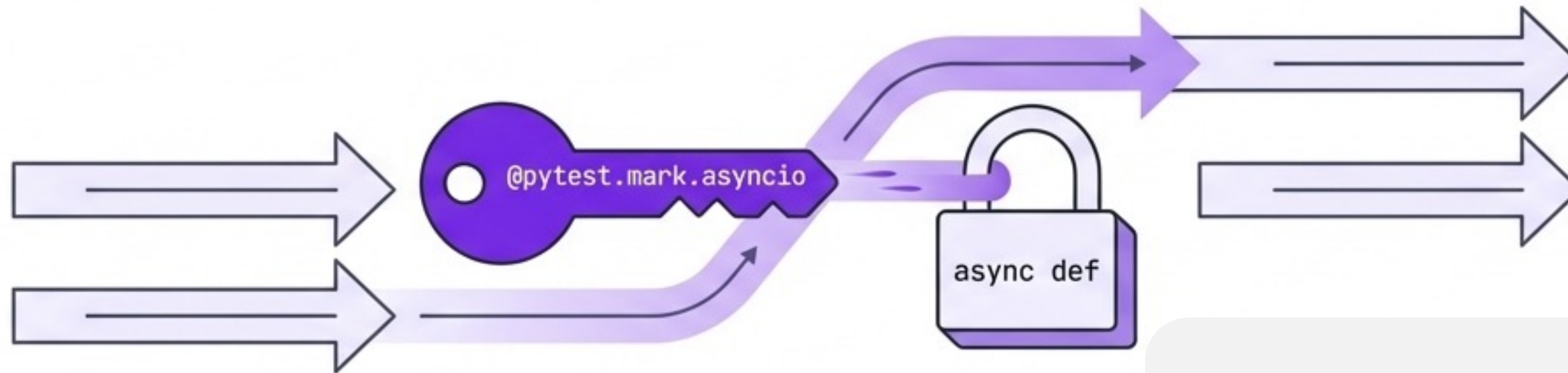
===== tests coverage =====
_____ coverage: platform linux, python 3.10.13-final-0 _____

Name                Stmts  Miss  Cover   Missing
-----
order_service.py      37      0   100%
TOTAL                 37      0   100%

===== 14 passed in 0.20s =====
```

6단계 : 모던 웹의 핵심, 비동기(Async) 함수 테스트

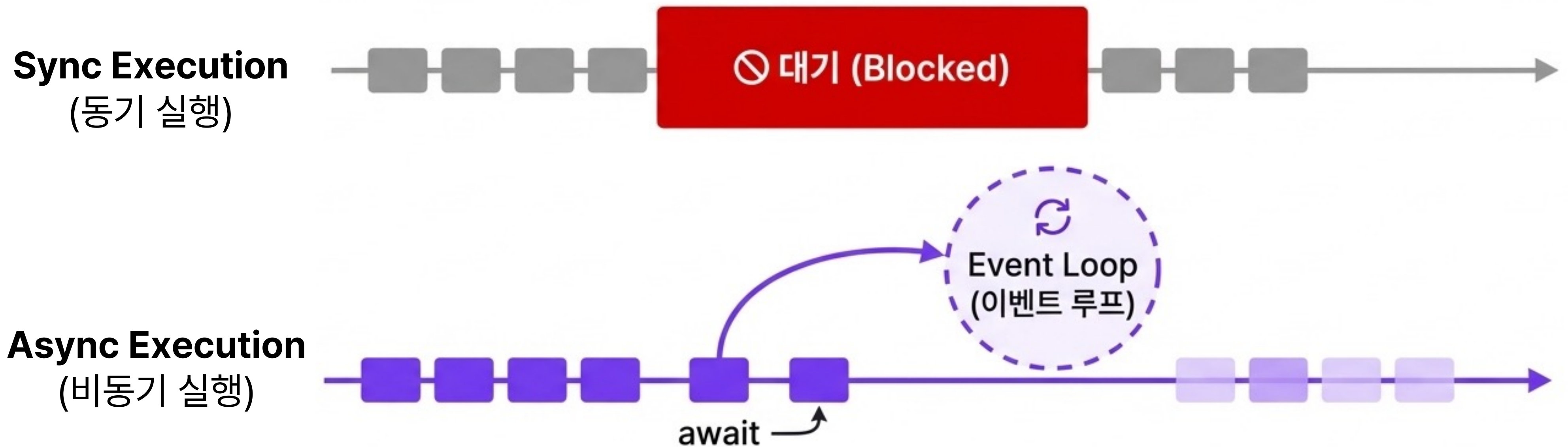
FastAPI 등 최신 비동기 웹 프레임워크 로직을 테스트할 때 필수적인 기술



- pytest-asyncio 설치 필수
- 복잡한 `asyncio.run()` 설정 없이, 테스트 함수 내부에서 직접 `await`를 즉시 사용할 수 있도록 지원

```
@pytest.mark.asyncio
async def test_fetch_data():
    result = await fetch_api()
    assert result == "success"
```

왜 비동기 전용 테스트 도구가 필요한가?



일반 테스트 환경은 이 '**제어권 양보(yield)**' 메커니즘을 이해하지 못함
pytest-asyncio가 테스트를 위한 **이벤트 루프**를 만들어줌

비동기 테스트 문법 (pytest-asyncio)

동기식 (Sync) 테스트

```
def test_sync_fetch():  
    data = fetch_data()  
    assert data is not None
```

일반적인 순차 실행 로직

비동기식 (Async) 테스트

```
@pytest.mark.asyncio  
async def test_async_fetch():  
    data = await fetch_data_async()  
    assert data is not None
```

pytest-asyncio 필요
비동기 함수 검증을 위한 필수 마커 부착

비동기 테스트의 핵심: 원리는 완벽하게 동일

1. 입력 준비 (Prepare)

2. 함수 실행 (Execute)

3. 결과 검증 (Verify)

비동기 테스트임을 선언

```
@pytest.mark.asyncio
async def test_get_greeting():
    result = await get_greeting("Elice")
    assert result == "Hello, Elice!"
```

달라지는 것은 오직
await 실행 환경

완벽한 테스트 코드를 위한 Checklist



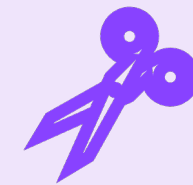
결과 검증 : 파이썬 기본
assert 문법 활용



예외 검증:
with pytest.raises()



데이터 반복:
@pytest.mark.parametrize



외부 고립: mock.patch
활용



품질 측정: pytest-cov
커버리지 확인

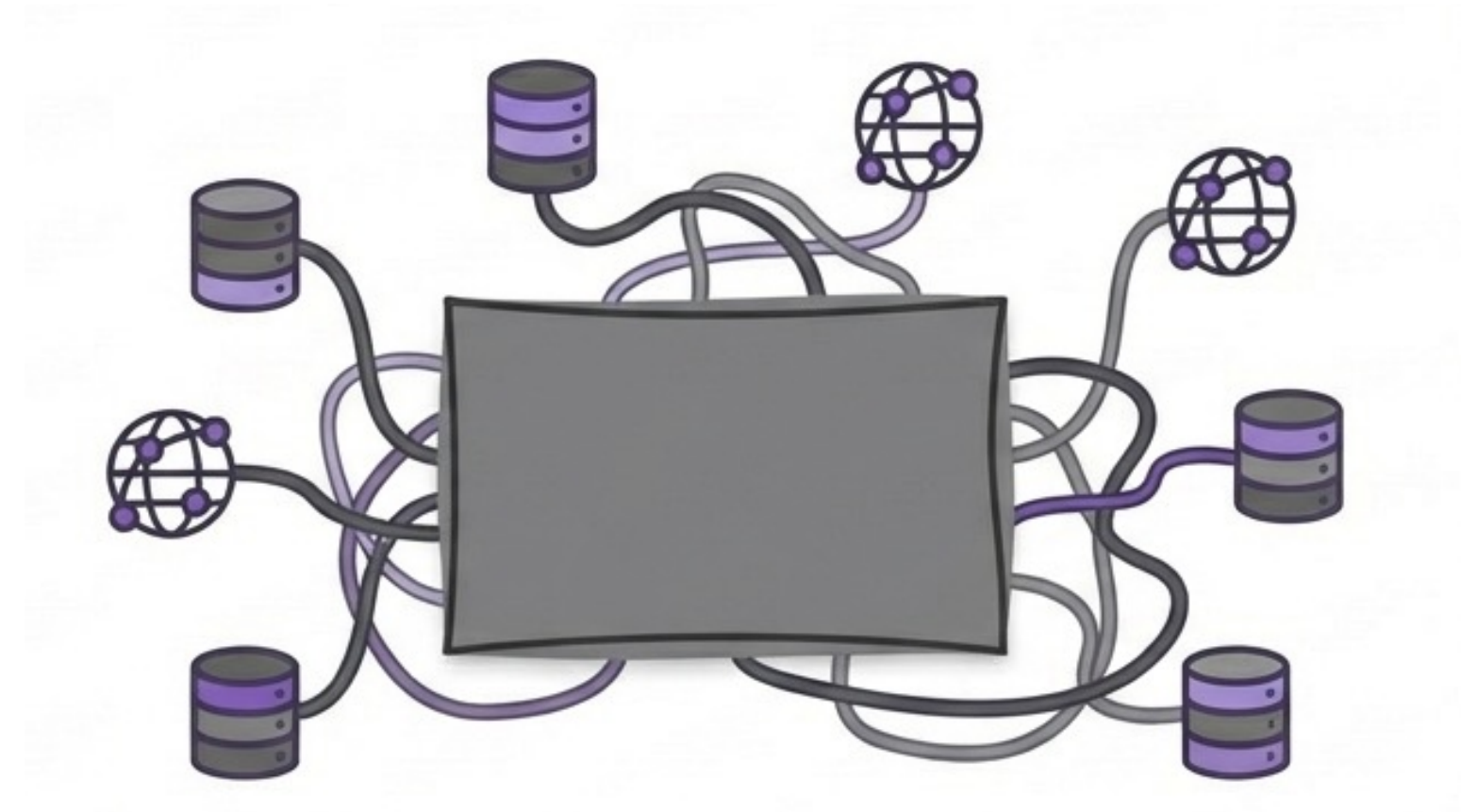


비동기 처리: pytest-
asyncio 적용

모든 코드가 똑같이 테스트하기 쉬운 것은 아님



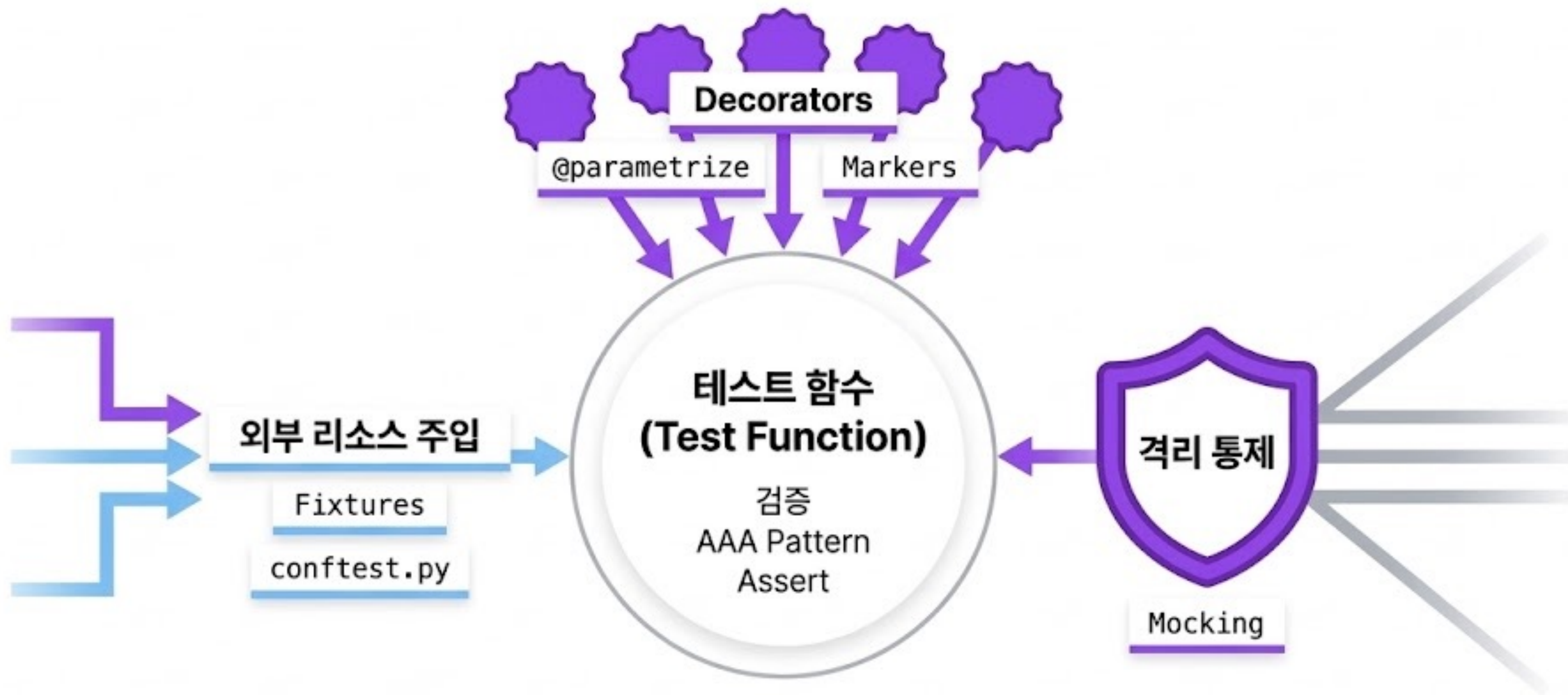
입출력이 명확한 함수 (순수 함수)



외부 의존성이 섞인 함수

테스트는 단순히 검증 도구일 뿐 아니라,
코드를 더 나은 구조로 나누게 만드는 '설계의 기준'이 됩니다.

pytest 문법 마스터 맵



테스트 주도 설계(Test-Driven-Design)로의 진화

